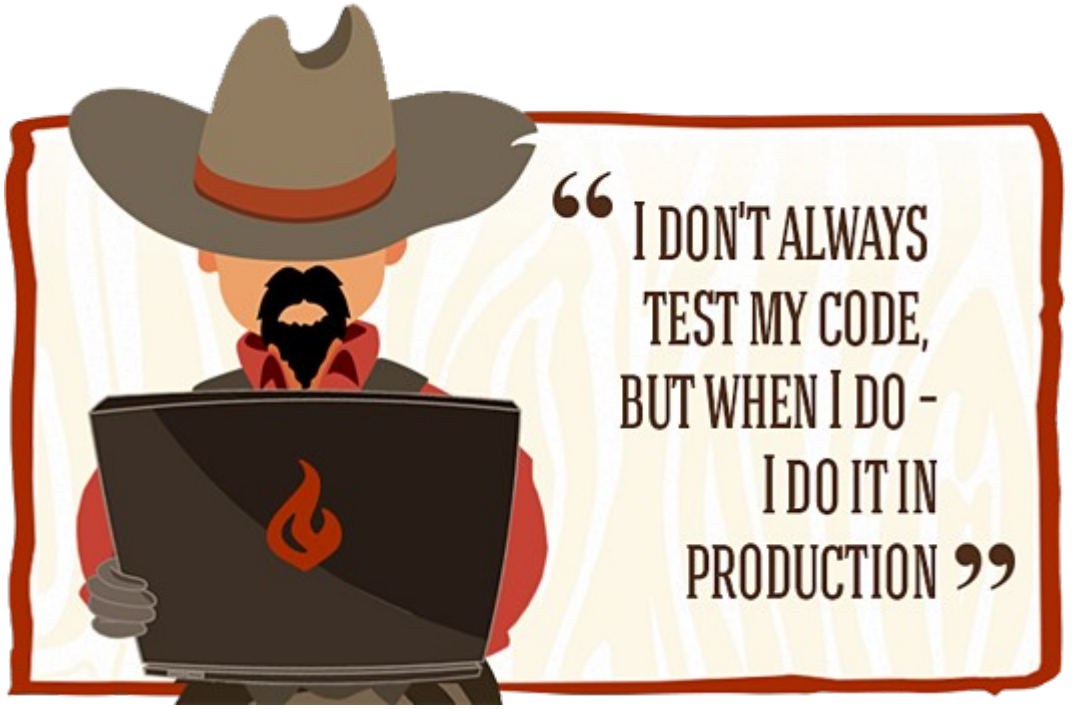
embedded and real-time systems

Unit Testing and Test-Driven development

SETR 26/27



The concept of “being in production”

- Software being "**in production**" means it's the final version that's been released and is **actively used by real end users.**
  - Production environment: where the live application resides. It's what users access and interact with.
  - Going into production: the process of deploying the software from a testing or staging environment to the live environment.
  - Testing vs. Production: **Before going into production**, software undergoes **rigorous testing** in an **isolated environment** to ensure stability and functionality.
- Thus, **software in production** is the **culmination** of **development and testing** efforts. It's the **working version that users rely on**.

Test Driven Development

- **TDD = Test Driven Development**
  - A software development process to develop code incrementally [1]

- **Principle: "Tests should drive the development of code"**
  - Tests are defined before the code is written
  - Code is accepted when it passes the predefined tests
  - The only necessary code is the code to make tests pass → all other code is superfluous

- Kent Beck is credited with the formalization of the concept [2], [3].

Basic rules of test-driven development

Write a test,

make it compile, make it link, make it fail

make it pass

refactor,

repeat.

Refactoring: modifying the structure of a given piece of code while preserving its behavior and functionality.

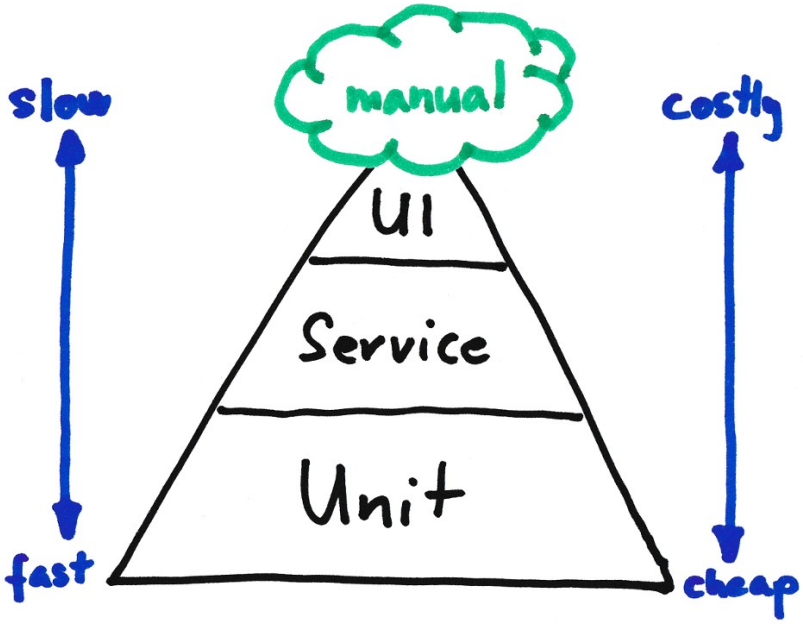
The objective is to “improve the code”, e.g. improve readability, efficiency, reduce code repetitions (organize code in functions/macros), etc.

See: http://www.c2.com/cgi/wiki?WikiPagesAboutRefactoring

Unit Test

- **Testing small units of code**
- **Unit: the smallest testable part of any software**
  - Usually, one or few inputs
  - One single output
  - Procedural programming:
    - Unit can be a **module**, **a function or procedure**
  - Object-oriented programming:
    - Unit usually is a class or an individual method
- **Unit testing should (must) be fast and automatic**

The testing pyramid



Cohn, Mike. Succeeding with Agile: Software Development Using Scrum. The Addison-Wesley Signature Series A Mike Cohn Signature Book. Upper Saddle River, NJ: Addison-Wesley, 2010.

Unit Test

- **Anatomy of a unit-test:**
  - Setup conditions
    - Assign values to variables, …
  - Perform test
    - Run unit under test
  - Check results

Unit Test

- **Not a unit test when:**
  - It talks to a database
  - It communicates across the network
  - It touches the file system
  - It can't run at the same time as any of your other unit tests
  - You have to do special things to your environment (such as editing config files) to run it.

http://www.odd-e.com/material/2009/JAOO_TDD_C_Tutorial/TDD_in_C_JAOO_tutorial.pdf

Why Unit-Testing?

- **Increase confidence in software**
  - Basic features are extensively tested (code automation allows large number of tests)
  - Regression is immediately noticed
    - Assure that code changes don't adversely affected existing features
- **Code re-usability improved**
  - Unit testing forces the creation of modular code
    - Specially important in Embedded Systems
- **Cost of fixing defects is reduced**
  - Problems are detected at an early stage
  - Cost of correcting at User Interface or System tests can be extremely high
- **Problem sources are more easily traced**
  - Unit test indicates clearly which part of the code generates the problem

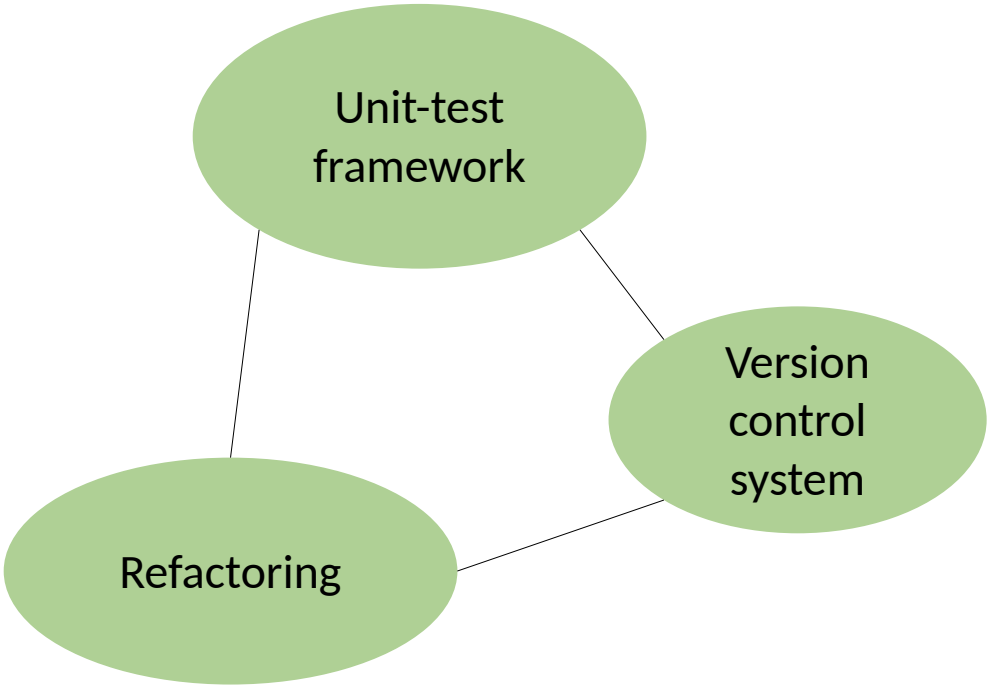
How to realize Unit-Test

Unit Test Requirements

- **Unit-test frameworks should:**
  - Allow creating tests easily and with small effort
  - Separate tests from production
- **Should be used a version control system**
- **Unit-test should be performed frequently**

Basic Unit-Test “Eco-system”

- **unit-test framework**
  - create your tests
- **version control system**
  - TDD is incremental: save every working increment
- **refactor**
  - first, make your test work
  - (save it)
  - then, improve code



What is a Unit

- **For C, there are no language constructs that define a unit.**

- **For Unit Testing we consider a unit a C module made up of:**
  - A C **header file** that contains function prototypes defining the interface to the C module.
  - An associated C **source file** that implements the functions and use file scope declarations to hide the internal details of the module.
  - A unit should provide some well defined functionality. E.g.
    - Manage a UART (init, send char, receive chars, …)
    - Manage IO ports (init, set as input/output, set/read digital value, …)
    - Manage a clock (init, increment, read, …)

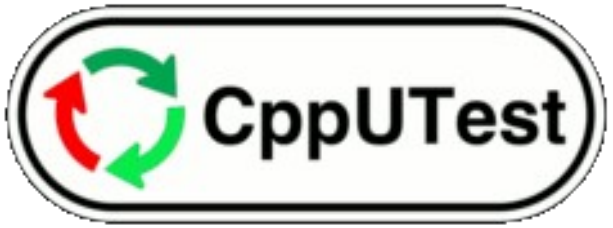
Unity

- **Unity** is a unit test framework written entirely in the C language.
- It is lightweight and possesses special features for embedded systems.
- Unity has scaled well from small to large embedded projects but can be used for any C project.
- At only one source file and a couple of header files, it can't get much smaller.

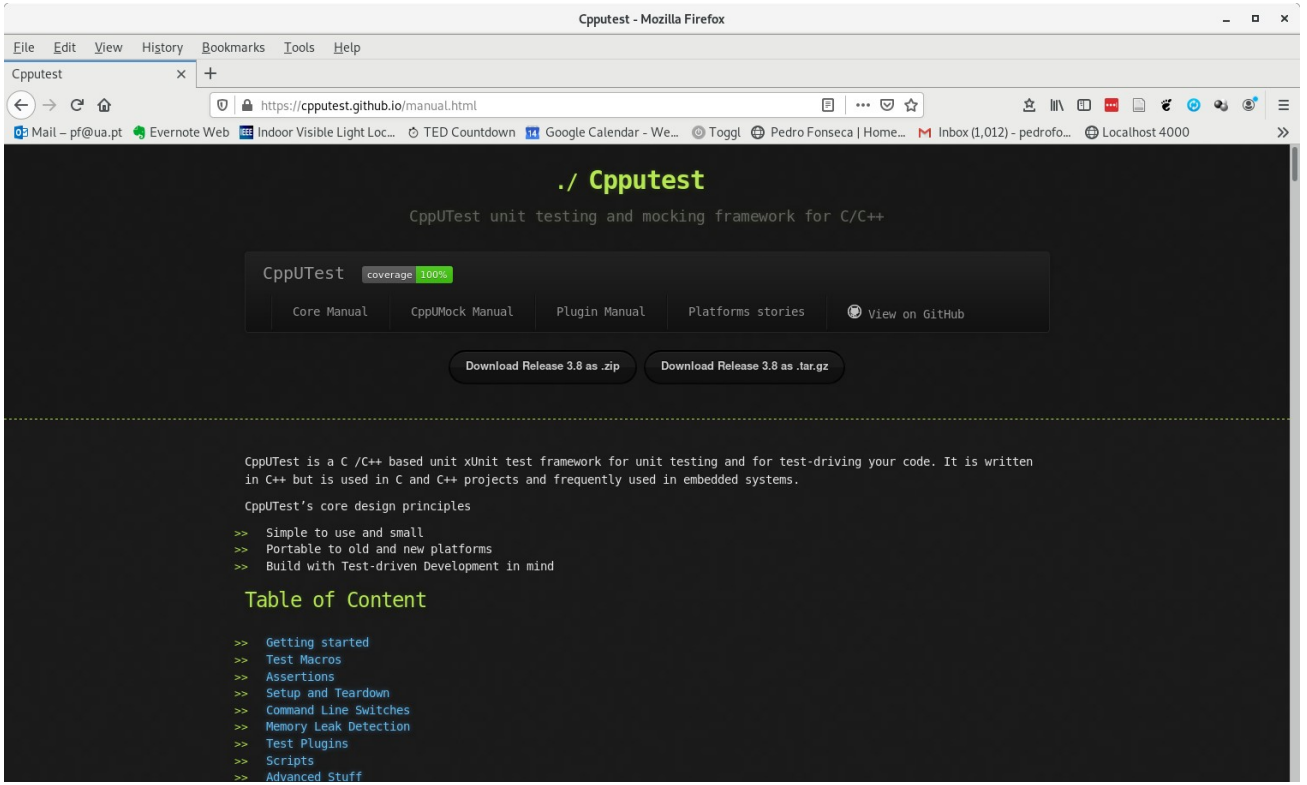
- http://throwtheswitch.org/



CppUTest



- www.cpputest.org
- Framework for testing C and C++ programs
- Developed for embedded systems



Unity

Unity (http://www.throwtheswitch.org/unity)

- **"UNIT TESTING FOR C (ESPECIALLY EMBEDDED SOFTWARE)" (from the website)**
  - 100% written in C code
  - Supports from 8-bit microcontrollers to 64-bit microprocessors
  - Can be executed natively on your host system (development PC), in a simulator or in the target
  - Can be obtained for free:
    - https://github.com/ThrowTheSwitch/Unity/archive/master.zip
    - git clone https://github.com/ThrowTheSwitch/Unity.git

Unity

- **Test structure**
  - The smallest realistic Unit Test build comprises:
    - A module under test (.c and .h)
    - A test file (c program),
    - and Unity (.c and .h files)
  - Compile all three and link them together.

  - Typically code is organized as follows:
    - Base folder | - makefile and generated executable
      | -- “src” sub-folder: unit source code (to test)
      | -- “test” sub-folder: test code
      | -- “build” sub-folder: executables

    - Note the “out of the tree” structure: source code, test code and compiler artifacts are in separate folders

Unity

- **Test program structure**
  - The test code is organized as a set of test functions followed by a main() that calls them

```
#include "unity.h"
#include "UNIT_UNDER_TEST.h"

void test_MyFunction_TestSet1(void)
{
      TEST_ASSERT_EQUAL_HEX8(10, MyFunction(10, 20, 30));
      ...
      TEST_ASSERT_EQUAL_HEX8(10, MyFunction(20, 30, 10));
}

void test_MyFunction_TestSet2(void)
{
      TEST_ASSERT_EQUAL_HEX8(108, MyFunction(126, 108, 110));
      ...
      TEST_ASSERT_EQUAL_HEX8(126, MyFunction(108, 110, 126));
}

// More tests
```

```
void setUp(void)
{
      /* Executed before tests. Can be used e.g. */
      /* for variable initializations */
      return;
}

void tearDown(void)
{
      /* Executed after tests. Can be used e.g. */
      /*  for resource cleanup */
      return;
}

int main(void)
{
      UNITY_BEGIN();
      RUN_TEST(test_MyFunction_TestSet1);
      RUN_TEST(test_MyFunction_TestSet2);
      ...
      return UNITY_END();
}
```

Unity

- **Test structure**
  - A test file which contains the tests:
    - Functions, arguments and output
  - Each test usually has multiple **assertions**(*).
  - If any of the assertions fails, that particular test fails and test execution continues at next test.
  - When completed, the results are displayed.

(*) an **assertion** is a boolean expression placed inside a test that verifies whether a specific condition holds true.

- **Compiling**
  - Compiling the test involves compiling the module under test, the test itself and unity
  - Can be something like:

```
$gcc TestFile.c MyModule.c ./unity/src/unity.c -o TestFile
$./TestFile
```

    - Note that this command line must be adapted to the particular folder structure in use

Unity

- **The output can be something like:**

```
testFile.c:21:test_MyFunction_TestSet1:PASS
testFile.c:22:test_MyFunction_TestSet2:PASS
-----------------------
2 Tests 0 Failures 0 Ignored
OK
```

- **Automating the process**
  - Tests compilation and execution should be made frequently
  - Compilation line can be longer (folders, compiler options, …)
    - Use of a makefile (or Cmake) is highly recommended

Unity

- **Lets take an example**
  - Module to handle a numeric vector
  - Partial implementation. Just functions to:
    - Init
    - Add values
    - Find a value and return its position
    - Returns the length of the vector
    - There are errors …

- **Lets look at the code …**

Unity

- **Homework**
  - Complete MyVectLib with:
    - Function to remove one element from the vector
    - Function to sort the vector
    - Functions to compute aggregate values (e.g. avg, max, min, …)
  - **And, of course, use Unity to check all these functions**

Unity

- "**Unity is mainly a rich collection of assertions and the support to gather up and easily execute those assertions.**" (Unity Assertions Reference)
  - An assertion establishes truth (i.e. boolean True) for a single condition. Upon boolean False, an assertion stops execution and reports the failure.
  - Unity allows to separate test assertions from source code (user and test code are separated).
  - There are many types of assertions, to handle different C types and assertion cases.

Unity

- The convention of assertion parameters generally follows this order:

TEST_ASSERT_X({modifiers}, {expected}, actual, {size/count})

  - **Actual**: value being tested, always present
  - **Modifiers**: masks, ranges, bit flag specifiers, …
  - **Expected**: the expected value/output to compare to an actual value. Some assertions don't need it (e.g. null check).
  - **Size/count**: refers to string lengths, number of array elements, etc.

- All assertions have a variant that includes a simple string message as a final parameter that is appended to an assertion failure message in Unity output:

TEST_ASSERT_X_MESSAGE({modifiers}, {expected}, actual, {size/count}, message)

Unity

- **Examples:**
  - Basic Fail, Pass and Ignore
    - **TEST_FAIL() / TEST_FAIL_MESSAGE("message")**
      - Used within conditional blocks. Test terminates and fails
    - **TEST_PASS() / TEST_PASS_MESSAGE("message")**
      - Aborts the remainder of the the test and returns pass. Used e.g. for tests with #ifdefs
  - Boolean
    - **TEST_ASSERT (condition)**
      - Pass if condition true, Fail otherwise
    - **TEST_ASSERT_TRUE (condition)** and **TEST_ASSERT_FALSE (condition)**
      - Pass/fail if condition matches the suffix

Unity

- **Examples:**
  - Pointers
    - **TEST_ASSERT_NULL (pointer)**
    - **TEST_ASSERT_NOT_NULL (pointer)**
      - Verify if a pointer is or is not NULL
    - **TEST_ASSERT_EMPTY (pointer)**
    - **TEST_ASSERT_NOT_EMPTY (pointer)**
      - Verify if the first element **dereferenced** from a pointer is or is not zero. Useful e.g. for null-terminated C strings.

Unity

- **Examples:**
  - Signed and Unsigned Integers

    TEST_ASSERT_EQUAL_INT (expected, actual)

    - Variant: INT, INT8, INT16,INT32_INT64 and unsigned prefixed with U (UINT, UINT8, …)
    - Variant: HEX assertions (HEX, HEX8, …)
      - Identical in function to unsigned integer assertions but produce failure messages with the expected and actual values formatted in hexadecimal.
  - **TEST_ASSERT_EQUAL_CHAR (expected, actual)**
    - For chars. Displays the char (if printable) or hex ASCII code otherwise

Unity

- **Examples:**
  - Masked and Bit-level Assertions
    - Masked and bit-level assertions produce output formatted in hexadecimal. Unity output is big endian.
  - **TEST_ASSERT_BITS (mask, expected, actual)**
    - Only compares the masked (i.e. logical high) bits of expected and actual parameters.
  - **TEST_ASSERT_BITS_HIGH (mask, actual)**
    - Asserts the masked bits of the actual parameter are logically high.
  - **TEST_ASSERT_BITS_LOW (mask, actual)**
    - Asserts the masked bits of the actual parameter are logically low.

Unity

- **Examples:**
  - And many others. There are assertions to test:
    - **Integer ranges**:
      - E.g. TEST_ASSERT_INT8_WITHIN (delta, expected, actual)
    - **Null-terminated strings**:
      - E.g. TEST_ASSERT_EQUAL_STRING (expected, actual)
    - **Memory blocks**:
      - E.g. TEST_ASSERT_EQUAL_MEMORY (expected, actual, len)
    - **Arrays**:
      - E.g. TEST_ASSERT_EQUAL_INT_ARRAY (expected, actual, num_elements)
    - Etc.

The build method

- **Unity can be executed:**
  1) On the target (a microcontroller in our case)
  2) On a simulator
  3) On the host (development) system

1) **On the target system** (often the worst option for embedded systems)
   - There is no full control over the registers, so injecting errors, which is essential for unit testing in Embedded Systems, can be hard of even impossible.
     - It is not possible to write to read-only registers nor read from write-only registers.
   - Downloading the firmware to the target takes a lot of time
   - Used complementarily to other methods, at late stages, e.g. to address platform-dependent issues

The build method

2) On a simulator

- Many embedded targets have simulators. **Registers, peripherals, and memory are usually treated as memory locations in RAM**, which for Unit Testing is quite convenient
  - E.g. assume a UART has a register USART1_STATUS that has a bit USART_STATUS_OVERFLOW.
  - In a simulator you can manipulate USART1_STATUS whenever you want to “fake” overflows and verify how the code responds.
  - These kind of tests are hard to force in real hardware, and it is even harder to replicate exact sequences.

The build method

3) On the host/development system

- Called “native executables”. Test are built and executed directly on Windows/Linux/MacOS/ using gcc, clang, …
  - **Test code that interacts with hardware/peripherals requires preparation**
  - Real addresses can’t be used. E.g. if a GPIO controller is at address 0x5000C000 in a microcontroller it is not possible to simply create a pointer to this location on a PC and write/read to/from it.

Microcontroller

```
#define PORTG_CONFIG (*(volatile unsigned int*)(0x5000C000))
#define PORTA_CONFIG (*(struct PORT_CONFIG_TYPE*)(0x50000000))
```

PC

```
volatile unsigned int PORTG_CONFIG;
volatile struct PORT_CONFIG_TYPE PORTA_CONFIG;
```

Homework

- **Implement a clock**

```
typedef struct {
    uint8_t   hundredths;
    uint8_t   seconds;
    uint16_t  minutes;
} swClock_t;

swClock_t swClock;
```

The contents of swClock are modified by two functions:

- `resetSwClock()`, that will set all fields to zero;
- `updateSwClock()`, to be called at every 10 ms to update the clock value.

Tests to be performed

- **For resetSwClock():**
  - set the value of all counters to zero

- **When updateSwClock() is called:**
  - Increment hundredths by one, and check for wrap-around
  - If necessary increment seconds and check for wrap-around
  - If necessary increment minutes

The whole figure …

- **Testing has several stages and objectives**
- **Unit testing is VERY important, but it is only a SMALL PART of the whole process, which involves:**
  - **Unit Testing**: testing very small pieces of code
  - **Integration Testing**: code doesn't break someone else's code
  - **System Testing**: exercising the entire application to make it fail
  - **Stress Testing**: push the limits of the application until it fails
  - **Beta Testing**: giving copies of the unfinished code to outside testers for feedback
  - **Acceptance Testing**: formal verification of requirements

Bibliography

1) J. W. Grenning, Test-driven development for embedded C. Raleigh, N.C.: Pragmatic Bookshelf, 2011.

2) K. Beck, Smalltalk best practice patterns. Upper Saddle River, N.J: Prentice Hall, 1997.

3) K. Beck, Test-driven development: by example. Boston: Addison-Wesley, 2003.

4) B. Vodde, 'CppUTest Core Manual | CppUTest'. [Online]. Available: http://www.cpputest.org/ . [Accessed: 04-May-2012].

5) 'List of unit testing frameworks - Wikipedia, the free encyclopedia'. [Online]. Available: http://en.wikipedia.org/wiki/List_of_unit_testing_framework s. [Accessed: 28-Apr-2013].

6) Piet Cordemans, Sille Van Landschoot, and Jeroen Boydens, 'Test-Driven Development for Embedded Software', 29-Sep-2011. [Online]. Available: http://ep.khbo.be/TDD4ES

7) Bas Vodde, 'Test-driven Development in C (and embedded)', 2009. http://www.odd-e.com/material/2009/JAOO_TDD_C_Tutorial/TDD_in_C_JAOO_tutorial.pdf

8) Grenning, J. (2007). Applying test driven development to embedded software. IEEE Instrumentation & Measurement Magazine, 10(6), 20–25. https://doi.org/10.1109/MIM.2007.4428578

9) http://www.throwtheswitch.org/unity (Accessed Apr 4, 2022)

10) Unity Test, Copyright © 2007 - 2021 Unity Project by Mike Karlesky, Mark VanderVoord, and Greg Williams